

Cover Letter — Thndr (Senior Backend Engineer, Money Movements Squad)

Headline — *LinkedIn note to the hiring team / email opener:*

I replaced a distributed-transaction design that had forced an entire enterprise platform onto one shared database with Temporal Saga orchestration — the same correctness-under-partial-failure problem money movement has. Plus real payments integration: Stripe, Paymob, webhooks with HMAC verification.

Moamen Moustafa Basyoni Alexandria, Egypt · +20 102 786 8067 · mmoamen270@gmail.com · github.com/moamen270

Dear Hiring Team,

I'm applying for the Senior Backend Engineer role on the Money Movements squad. What drew me to it is the emphasis on correctness — deposits, withdrawals, settlement and reconciliation are the class of problem where "mostly works" is a failure, and that's the problem I've spent the last two years on.

The specific parallel: the platform I work on at Andalusia ran distributed transactions through MS DTC, which isn't supported on .NET Core and had been worked around by forcing every service onto a single shared database. I replaced that with **Temporal Saga orchestration** — long-running, recoverable transactions with compensating steps across service boundaries — and improved 10+ inter-service interaction patterns in the process. That is the settlement and reconciliation problem in a different domain.

Also relevant to your stack and scope:

- Backend in **both C# / .NET Core and Python** — ASP.NET Core and Minimal APIs on one side, FastAPI and Flask with SQLAlchemy and async/await on the other.
- **PostgreSQL and SQL Server** via Entity Framework Core; **RabbitMQ** for event-driven communication.
- **Payments integration:** Stripe and Paymob at Trastain and MYM, including webhook processing with HMAC signature verification and transaction security for booking execution.
- An OpenID Connect / OAuth2 identity server with JWT and scope-based authorization, plus OWASP-aware coding and PII masking.
- Production ownership — I troubleshoot live issues and performance-tune; a runtime rule engine I built runs 45% faster, and an API gateway change cut deployment downtime from two hours to fifteen minutes.

To be straightforward on two things: I'm at roughly three years rather than a senior track record, and I haven't worked in a regulated financial domain — banks, settlement rails and reconciliation would be new to me. The distributed-correctness engineering underneath them would not be, and I'd rather say that plainly than oversell it.

I'd welcome the chance to talk.

Best regards, Moamen Basyoni